

Mythos

Sistema gastronómico multi-restaurante · Auditoría integral v1.0

Análisis completo del sistema

Estado actual · módulos por panel · bugs detectados · roadmap a producción

Restaurante demo

La Huaca — Asunción, Paraguay

Stack

React 18 (CDN) + Babel Standalone · Supabase (PostgreSQL + Auth + Realtime + Storage + RLS) · Vercel estático

Cobertura del análisis

- 9 paneles HTML (~25.000 líneas)
- 73 migraciones SQL
- ~25 tablas Supabase
- 1 Edge Function (gestión de usuarios)
- Flujos: cliente QR, delivery, mozo, cocina, caja, admin, gerente, superadmin

Veredicto rápido

Mythos es un sistema sorprendentemente completo para una v1: cubre el ciclo completo cliente !' cocina !' mozo !' caja con paneles adicionales para administración, supervisión, multi-restaurante y delivery. El producto está en ~70 % de "listo para producción seria". Los huecos que separan el demo de un SaaS estable son tres: (1) seguridad RLS multi-tenant, (2) pasarela de pagos real (Bancard), (3) automatizaciones de operación (impresión, WhatsApp, Google APIs).

Complejidad estimada · 70 % (MVP avanzado, listo para piloto controlado, no para flota)

Índice

1. Resumen ejecutivo y veredicto
2. Arquitectura y stack técnico
3. Componentes compartidos entre paneles
4. Panel Cliente QR ([index.html](#))
5. Panel Cliente Delivery ([delivery-cliente.html](#))
6. Panel Rider ([delivery-rider.html](#))
7. Panel Cocina · KDS ([cocina.html](#))
8. Panel Mozo ([mozo.html](#))
9. Panel Caja ([caja.html](#))
10. Panel Administrador del local ([admin.html](#)) — 18 submódulos
11. Panel Gerente / Supervisor local ([gerente.html](#))
12. Panel Superadmin · plataforma SaaS ([superadmin.html](#))
13. Base de datos · esquema, RLS y migraciones
14. Bugs detectados · catálogo priorizado
15. Cómo descubrir más bugs · metodología
16. Seguridad · riesgos y plan de mitigación
17. Próximas integraciones (Bancard, APIs, Google, WhatsApp...)
18. Mejoras · cambios · obsoletos
19. Roadmap a producción real
20. Crítica final y % de completitud detallado

1 · Resumen ejecutivo

Mythos es un ecosistema SaaS gastronómico multi-restaurante construido sobre un patrón intencional de “React por CDN + Babel Standalone + Supabase”: cada panel es un archivo HTML autocontenido con su propio estado, sin bundler, lo que reduce fricción de despliegue y permite iterar muy rápido. El sistema está preparado para multi-tenant (columna restaurant_id por tabla, panel superadmin completo, soporte chat Admin/Gerente !” Superadmin), pero todavía no se comporta como multi-tenant porque la mayoría de políticas RLS usan USING(true), lo que en producción real expondría datos entre restaurantes.

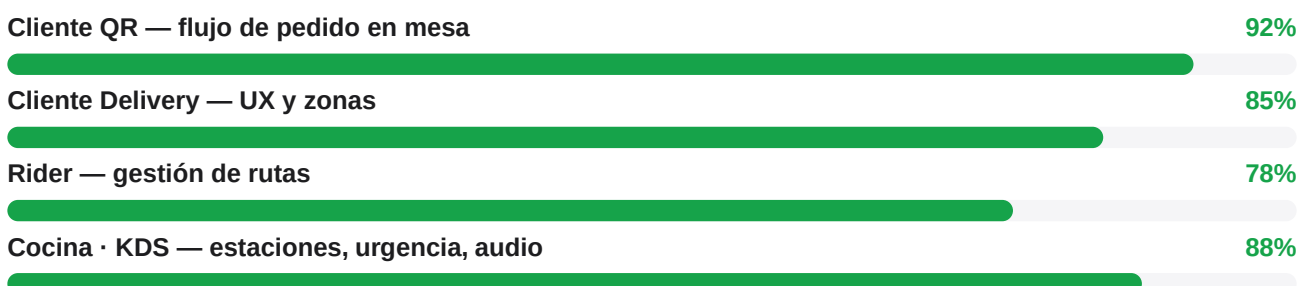
Lo que sorprende positivamente

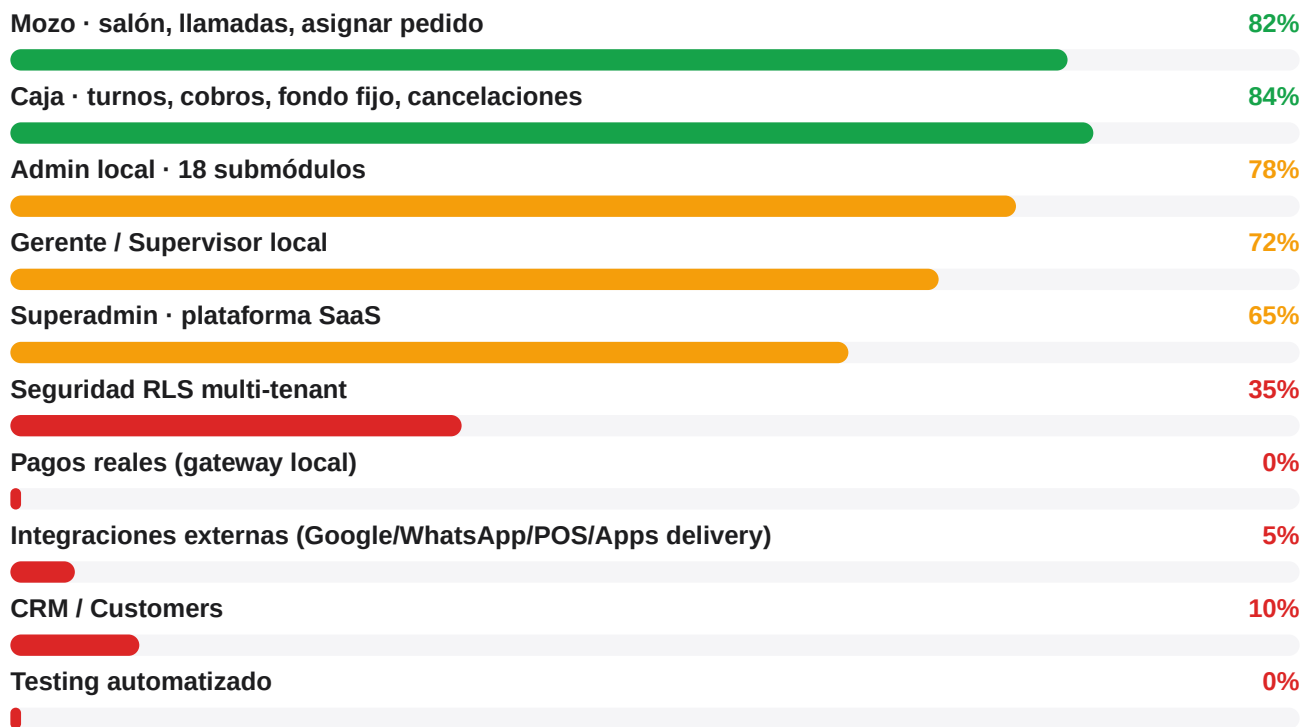
- Cobertura de roles muy completa: cliente QR, cliente delivery, rider, mozo, cocina, caja, admin, gerente, superadmin.
- Flujo de caja con turnos reales (apertura/cierre, fondo fijo configurable, retiro automático del excedente, cancelaciones, quejas, ingresos/egresos).
- Sistema de cocina con estaciones de despacho independientes por categoría + zona, con token compatible por estación.
- Reservas migradas de localStorage a la tabla reservations, con zona preferida.
- Edge Function /api/create-user.js: el service_role ya salió del frontend (riesgo de seguridad crítico resuelto).
- Tracking del cliente con Realtime + polling de respaldo cada 10s y reconexión en visibilitychange — robusto en iOS.
- Soporte 1-a-1 Gerente/Admin !” Superadmin con metadata automática (canal, restaurante, autor).
- Diseño visual consistente (Apple-like, tipografía system, sombras suaves) sin un design system pesado.

Lo que falta para llamarlo “listo para flota”

- RLS real multi-restaurante: hoy 112 políticas con USING(true) en 24 migraciones.
- Pasarela de pagos local (Bancard / Tigo Money / Pago Móvil) — hoy el pago en el cliente es declarativo.
- Login por PIN del rider sin rate limiting ni hash — riesgo de fuerza bruta.
- Integración con impresoras térmicas y comandera física (ESC/POS).
- Salidas a aplicaciones de delivery (PedidosYa, Bolt Food, Rappi) o al menos webhook genérico.
- WhatsApp Business o Twilio para confirmar pedidos/reservas y enviar QR del ticket.
- CRM/Customers — hoy los clientes no se persisten como entidad consultable; van como snapshot dentro de orders.
- Pruebas automatizadas: el sistema no tiene tests; toda la validación es manual.

Distribución de completitud por área





Lectura del estado general

Mythos es claramente un MVP avanzado, no un sistema demo. Tiene volumen real de código, persistencia cuidada y flujo operativo completo. La distancia con un producto SaaS comercial es operativa (pagos, WhatsApp, impresión, multi-tenant seguro) y no de funcionalidad de fondo. En un piloto controlado en La Huaca podría estar funcionando hoy con supervisión humana; para vender a otros restaurantes hacen falta las piezas críticas que faltan.

2 · Arquitectura y stack técnico

Visión general

No hay servidor propio: el frontend es estático en Vercel y todo el backend vive en Supabase. La única pieza serverless es una función Vercel (`api/create-user.js`) que actúa como puente cuando hace falta `service_role` (creación segura de usuarios). El cliente carga React desde CDN y compila JSX en el navegador con Babel Standalone — lo cual significa: cero build step, pero un costo de carga inicial que importa cuando el local tenga Wi-Fi inestable.

Pilares

Frontend	React 18 UMD (CDN) + Babel Standalone, CSS-in-JS inline. Sin bundler. <code>design-system.css</code> para variables compartidas.
Backend	Supabase: PostgreSQL + Auth + Realtime + Storage + Row Level Security.
Auth	Login unificado en <code>/login.html</code> con username (no email). Función SECURITY DEFINER <code>get_user_email</code> convierte username != email interno <code>@mythos.internal</code> .
Roles	superadmin · admin · supervisor_local (gerente) · cajero · mozo · cocina · rider.
Realtime	Suscripciones por canal en <code>orders</code> , <code>waiter_calls</code> , <code>delivery_orders</code> , <code>support_messages</code> .
Storage	menu-images (Storage bucket Supabase) para fotos de platos y portadas de restaurante.
Deploy	Vercel estático con <code>outputDirectory: public/</code> . PR/preview por push.
Configuración	<code>config.js</code> gitignored. <code>window.SUPABASE_CONFIG</code> con url, anonKey y <code>restaurantId</code> .
Serverless	<code>/api/create-user.js</code> — gestión segura de usuarios (admin de <code>auth.users</code> + <code>user_roles</code>).

Patrones intencionales que NO se deben tocar

- No introducir Vite/Next/Webpack: rompe el patrón "edición en caliente" que es la ventaja del proyecto.
- No usar `import/export`: todo el código vive en `window.*` o en scripts globales.
- Comentarios `/*EDITMODE-BEGIN*/` y `/*EDITMODE-END*/` son delimitadores del panel de tweaks visual; no borrar.
- `RESTAURANT_ID` fijo (00000000-0000-0000-0000-000000000001) durante el demo; en multi-tenant real vendrá del JWT/profile.

Diagrama mental del flujo principal

[cliente QR] != Supabase.insert(orders, order_items) != [tracking realtime] != [cocina KDS] (avanza status) != [mozo] (entrega, marca `delivered_to_table_at`) != [caja] (cobra, registra movimiento, libera mesa).
[delivery-cliente] != orders (order_type=delivery) + delivery_orders (rider_status) != [admin Delivery] (asigna rider) != [delivery-rider] (recoge, on_way, delivered).
[admin] (ABM) != Supabase.* · [gerente] supervisa, aprueba, registra incidencias · [superadmin] gestiona restaurantes, planes, suscripciones, pagos y soporte.

3 · Componentes compartidos entre paneles

Aunque cada panel es un HTML aislado, hay piezas que se repiten y deberían tratarse como contrato común. Documentarlas evita inconsistencias y facilita extraer un módulo común más adelante.

3.1 · Cliente Supabase (_initDB)

Dónde vive	En cada panel HTML. Función <code>_initDB()</code> que lee <code>window.SUPABASE_CONFIG</code> .
Limpia BOM	Hace <code>.replace(/^\p{}/, "").trim()</code> porque el Dashboard Supabase a veces inyecta caracteres invisibles.
Caída elegante	Si no hay config retorna null y los paneles entran a modo demo (con datos estáticos en <code>index.html</code> / <code>cocina.html</code>).
Mejora sugerida	Extraer a <code>/public/lib/db.js</code> para evitar 9 copias divergentes del mismo bloque.

3.2 · Login unificado (/login.html)

Auth	Supabase Auth con email/password, pero el usuario tipea un username.
Conversión	RPC <code>get_user_email(username)</code> — SECURITY DEFINER, accesible sin auth — devuelve email interno.
Redirect	<code>redirectByRole()</code> : superadmin !' superadmin.html, admin !' admin.html, supervisor_local !' gerente.html, cajero !' caja.html, mozo !' mozo.html, rider !' delivery-rider.html, cocina !' cocina.html.
Posible añadido	Recordar último usuario (localStorage), "olvidé mi contraseña" interno para el admin, 2FA opcional para superadmin.

3.3 · Diseño visual común

Paleta	#000 / #1D1D1F / #6E6E73 / #D2D2D7 / #F5F5F7. Acento #0A84FF. Estados: ok #16A34A, warn #F59E0B, danger #FF3B30/#DC2626.
Tipografía	system-ui · SF Mono para números. Headings 800. Body 13–14.
Toasts	Función <code>toast(msg, ok)</code> presente en admin, caja, mozo, gerente. Sin contrato unificado.
Modal	Componente Modal({title, onClose, children, width}) replicado en cada panel.
Mejora sugerida	Extraer Btn, Modal, Toast, Kpi, Badge, Inp, Sel a <code>design-system.js</code> (no romper el patrón CDN — basta un <code><script type="text/babel" src="/lib/ui.jsx"></code>).

3.4 · Realtime y reconexión

El patrón que mejor funciona — usado en `index.html` para tracking — es: suscripción a canal + polling de respaldo cada 10 s + reconexión en `visibilitychange`. Vale la pena estandarizarlo en todos los paneles que dependen de realtime (cocina, mozo, caja, rider). Hoy algunos solo se suscriben.

3.5 · Constantes que deberían estar en una sola tabla

- RESTAURANT_ID hardcodeado en cada panel — deberá venir del JWT cuando arranque multi-tenant real.

- TABLE_NUM por URL ?mesa=4. Pendiente: token único por mesa para QR real.
- METODOS_PAGO: efectivo, tarjeta_credito, tarjeta_debito, qr, mixto — repetido en caja y admin.
- SL / SC (status label/color de orders): repetido literal en gerente, mozo, caja, cocina.

4 · Panel Cliente QR (public/index.html)

Tamaño	1.724 líneas · React 18 + Babel.
Rol	Cliente final escanea QR en mesa y pide sin necesidad de mozo.
Persistencia	Inserta directamente en orders, order_items, order_item_extras, order_status_history. Coupon validado vía RPC. Reservation insertada en reservations.
Modo demo	Sin config funciona con menú estático (MENU_STATIC) y cupón MESA10 falso.

Pantallas

#	Pantalla	Función
1	QRScreen	Splash inicial, simula escaneo.
2	ProfileScreen	Selector idioma, comer aquí / para llevar, botón "llamar mozo", reserva.
3	MenuScreen	Categorías, ítems, badges de promo (pizza_corrida, tenedor_libre...).
4	ProductModal	Detalle con extras y observaciones.
5	CartScreen	Editar, eliminar, cupón, partir cuenta (SplitBillModal).
6	PayScreen	Método de pago + datos de factura (RUC/CI/correo).
7	TrackingScreen	Pasos en vivo (Realtime + polling fallback).
8	RatingScreen	Estrellas + razones + comentario.
9	ReservationScreen	Reserva con zona, ocasión, observaciones.

Qué se le puede agregar

- Login social opcional para clientes recurrentes (auth.signInWithOAuth Google) !' arranca el CRM real.
- Historial del propio cliente (últimos pedidos en este restaurante / en todos los Mythos).
- "Mi cuenta dentro de la mesa" (mesa compartida con varios celulares — items por persona dentro del mismo order_number).
- Idioma real: hoy lang viaja en orders.language pero los textos están en español; añadir i18n por archivo JSON.
- Modo offline-first: cachear menú con Service Worker (PWA) — clave en zonas con Wi-Fi débil.
- Carrito persistente en localStorage si se cierra la pestaña.
- Pagos reales con Bancard QR / Pago Móvil / Tigo Money (ver sección 17).
- Botón "ya estoy aquí" cuando hay reserva !' notifica al mozo automáticamente.
- Mejoras de accesibilidad: aria-labels, contrastes en modo claro, fuente +1 para personas mayores.
- Mostrar en tracking el nombre del mozo asignado y/o foto.

5 · Panel Cliente Delivery (public/delivery-cliente.html)

Tamaño	1.941 líneas.
Rol	Cliente que pide a domicilio desde la web pública (no QR).
Cálculo de zona	haversineKm() + calcDeliveryFee() con tabla delivery_zones (color, precio, distancia)
Canal	Parametro ?canal=web (también podría ser whatsapp, ig, telefono — se guarda en orders.channel).

Pantallas

- WelcomeScreen — elige delivery o pickup o reserva.
- CoverageScreen — ingresa dirección/ubicación, valida zona, muestra fee.
- CustomerDataScreen — nombre, teléfono, observación.
- MenuScreen / ProductModal / CartScreen — análogos al cliente QR pero con order_type=delivery.
- Tracking simplificado (sin sub-pasos cocina pero con estado del rider).

Mejoras sugeridas

- Google Maps Places Autocomplete + selector visual sobre el mapa !' precisión del domicilio.
- Geocoding inverso: pegar coordenadas !' dirección legible para el rider.
- Estimar ETA basado en distancia + tráfico (Google Distance Matrix).
- Login con número de teléfono + OTP (Supabase phone auth) — habilita historial de cliente.
- Cupones de delivery (descuento de fee si es la primera compra, por zona, por horario).
- Push notifications (PWA) cuando el pedido sale a la calle.

6 · Panel Rider (public/delivery-rider.html)

Tamaño	749 líneas.
Rol	Repartidor (interno o externo). Login por PIN numérico.
Datos	Tabla delivery_riders (rider_pin, current_status: disponible/en_ruta/offline, vehicle, foto).
Flujo	Login PIN !' Home (pedidos asignados) !' Iniciar ruta !' cambiar rider_status pickup!'on_way!'delivered.

Mejoras sugeridas

- Hash del PIN (bcrypt) + rate limiting + bloqueo tras N intentos — hoy el PIN se compara en texto plano.
- Geolocalización en vivo: al iniciar ruta, navigator.geolocation.watchPosition() !' upsert en delivery_rider_positions cada N segundos para que admin/cliente vea ubicación.
- Ruta optimizada multi-pedido (Google Directions API).
- “Llamar cliente” con tel: + mostrar referencia del domicilio.
- Foto del entregado (delivery_orders.proof_photo_url) — confirma entrega y zanja disputas.
- Estadísticas históricas (km, ingresos, propinas) más completas que las actuales.
- Modo bici/moto con velocímetro y registro de horas trabajadas (employee_shifts).

7 · Panel Cocina · KDS (public/cocina.html)

Tamaño	1.789 líneas.
Rol	Kitchen Display System. Vista kanban: nuevo / preparando / listo.
Realtime	Subscripción a INSERT y UPDATE en orders.
Estaciones	Filtra tickets por estación (kitchen_stations) — cada categoría puede estar asignada a una estación distinta. Link compartible por token.

Submódulos detectados

- TicketCard — temporizador, urgencia (verde/amarillo/rojo), badges (delivery, alergia, observación).
- StationTabs — pestañas por estación, conteo de tickets pendientes por cada una.
- OrderTypeFilterTabs — filtra todos / mesa / delivery / takeaway.
- StatsPanel — promedio de tiempo, top items, total atendido hoy.
- ConfigDrawer — sonido on/off, modo compacto, audio para alérgenos.
- FelicitacionesPanel — mensajes motivacionales (gamificación liviana).
- KitchenMessageBanner — banner editable desde admin para avisos al equipo.

Qué se le puede agregar

- Impresión automática del comanda al pasar a “preparando” (ESC/POS via QZ Tray o nube).
- Métricas SLA por estación (% tickets servidos en <X minutos).
- “Pausa de estación”: marcar 86 (sin stock) un plato y bloquearlo en el menú del cliente automáticamente — hoy 86 está parcialmente en gerente.html (item_86_list).
- Voz / TTS para alergias (lectura en voz alta).
- Vista de “línea” (mise en place) con conteo agregado de ingredientes por estación.
- Métrica de carga (tickets/min) y alerta de saturación.

8 · Panel Mozo (public/mozo.html)

Tamaño	3.139 líneas.
Rol	Mozo del salón. Ve mesas, ocupación, llamadas, asigna pedidos, marca entrega.
Salón visual	ZonaMozo dibuja el plano del local con zonas y mesas con coordenadas virtuales (vx/vy normalizados 0–100). Forma rectangular/redonda/cuadrada.
Sonidos / vibración	initAudio(), playAlertSound(), vibrateDevice() para llamadas.

Estados resueltos por mesa

getMesaStatus combina ordersMap, callsMap y reservationByTable para devolver: libre · ocupada · llamando · paid_pendiente · reservada (con ventana de alerta).

Mejoras sugeridas

- Dividir cuenta entre comensales (SplitBillModal del cliente, en mozo, persistido en DB).
- Recordatorio automático si una mesa lleva X minutos sin actualización (probable abandonada).
- Acción “transferir mesa” a otro mozo (turno cruzado).
- Vista “mis mesas” vs “todas” (cuando hay varios mozos en piso).
- Mini-chat con cocina (canal de waiter_calls extendido con type=chat).
- Tablero de propinas (employee_shifts.propinas) y resumen al final del turno.
- Login por PIN/QR del propio mozo (en lugar de username/password) para mayor velocidad.

9 · Panel Caja (public/caja.html)

Tamaño	3.549 líneas.
Rol	Cajero. Apertura/cierre de turno, cobros, cancelaciones, movimientos, quejas, retiros.
Persistencia	turnos_caja, movimientos_caja, cancelaciones_caja, quejas_sugerencias.
Fondo fijo	Configurable por restaurante (cash_mode_default, cash_fondo_fijo, cash_diff_umbral, cash_auto_retiro_excedente).

Submódulos

- AperturaTurnoScreen — abre turno con monto inicial (sugerido = fondo fijo si modo=fijo).
- CobrosPanel — lista pedidos pendientes de cobro. CobroModal con métodos: efectivo, tarjeta_cred/déb, QR, mixto. Calcula cambio y muestra denominaciones.
- CancelacionesPanel — registra anulaciones con motivo y aprobación.
- IngresosEgresosPanel — caja chica manual.
- QuejasPanel — registra queja/sugerencia que después aparece en Gerente.
- RetiroPanel — sangrías parciales del cajón.
- TomarPedidoPanel — caja también puede tomar pedido (modo mostrador / barra) con PagarAntesDeEnviarModal.
- SalonPanel — plano completo del salón con totales por mesa.
- CierreCajaPanel — arqueo final, calcula diferencia vs. esperado, exige justificación si excede cash_diff_umbral, genera retiro automático del excedente si está configurado.
- CalculadoraFlotante — herramienta para conteo de denominaciones.

Qué se le puede agregar

- Impresión real del ticket fiscal (Bancard / SET integration).
- Conciliación nocturna automática vs Bancard webhook.
- Lectura de QR del cliente para identificar mesa al cobrar (acelera CobroModal).
- Modo “mostrador” permanente con teclado numérico y atajos para barra.
- Historial de turno con búsqueda por nº de pedido / cliente.
- Bitácora visible: arqueo previo, observación del cajero entrante.

10 · Panel Administrador del local (admin.html) · 18 submódulos

El panel admin es el más grande del sistema (8.036 líneas). Concentra toda la administración del local. Lo desgloso por submódulo con su estado y mejoras propuestas.

Submódulo	Estado	Observaciones
Dashboard	OK	KPIs ventas, pedidos, ratings, tickets activos.
Pedidos	OK	Listado, filtros, detalle.
Menú	OK	Categorías, ítems, extras, ImageUploader.
Mesas	OK	Plano editable, zonas, forma, vx/vy.
Personal	OK	ABM users con Edge Function.
Clientes	parc.	Lista derivada de orders (no hay tabla customers todavía).
Caja	OK	Resumen de turnos cerrados, exportar.
Finanzas	OK	Reportes de ingresos / costos, expenses.
Marketing	OK	Cupones, métricas de uso.
Ratings	OK	Estrellas, comentarios, moderación pendiente.
Stock	OK	ingredients, recipes, movements, alertas.
Reportes	OK	Filtros de fecha, agrupaciones, export.
Estaciones	OK	Kitchen_stations, mapeo categoría!"estación, token compartible, audit.
Config	OK	Datos del restaurante, horarios, portada, ubicación.
Delivery	OK	Dashboard delivery, pedidos, riders, zones (MapEditor).
Reservas	OK	CRUD reservations, ventana de alerta.
Proveedores	NUEVO	Suppliers + contactos + compras (072).
Soporte	NUEVO	Chat con superadmin (073).

Hallazgos finos por submódulo

- Menú — falta versión “a granel” para activar/desactivar muchos ítems a la vez (86 list rápido). El ImageUploader podría comprimir en cliente antes de subir.
- Mesas — el MapEditor podría exportar/importar plano (JSON) y permitir snap a grid.
- Personal — no muestra historial de turnos por empleado fuera de employee_shifts; agregar “informe nómina”.
- Clientes — empezar a poblar customers (id, phone, email, n_pedidos, ticket_promedio) con un upsert al cerrar cada orders. Crítico para retención.
- Finanzas — separar “costo estimado por receta” real cuando recipes esté completo; hoy se acerca por margen plano.
- Marketing — falta segmentación (por zona, por horario, por cliente top); cupones por código QR físico.
- Ratings — moderación pendiente (la auditoria_contexto_ia.md lo marca explícito); aplicar visibilidad pública/privada.
- Stock — los movimientos automáticos al cerrar pedido (descuento de receta) deberían validarse con un test masivo.

- Delivery !' MapEditor — ya soporta polígonos por color. Faltaría preview en cliente mostrando si su pin cae en zona antes de pedir dirección exacta.
- Soporte — al cerrar ticket, mandar email/WhatsApp al admin con resumen.

11 · Panel Gerente · Supervisor local (gerente.html)

Tamaño

1.712 líneas.

Rol

Supervisor del piso. Toma decisiones que afectan al turno sin tocar configuración del restaurante.

Tablas que usa

shift_logs, manager_approvals, item_86_list, quejas_sugerencias, ratings, support_tickets, orders.

Módulos detectados

- Dashboard — KPIs del día, alertas, ticket promedio, productividad por mozo.
- SupervisionTurno — vista del salón en vivo, asistencia, propinas, transferencias.
- Aprobaciones — solicitudes de descuento, cortesía, anulación con motivos.
- QuejasYatings — vista de quejas y reseñas pendientes de revisión.
- Stock86 — marcar/desmarcar “sin stock” un plato en caliente (ítem_86_list).
- Bitácora — shift_logs (categorías: nota, incidencia, tarea, traspaso, cliente, personal, equipo, limpieza).
- Soporte — abrir/responder tickets al superadmin (chat).

Mejoras sugeridas

- Aprobaciones con notificación push al admin cuando se rechaza/aprueba.
- Reglas automáticas (ej. cortesía gratis si rating <2 con motivo “tiempo”).
- Indicadores de cumplimiento (cuántas incidencias se resolvieron en N horas).
- “Cierre de turno gerente”: snapshot legible de bitácora del día (PDF/Mail al admin).
- Sincronía con Stock86 !' cuando un ítem queda en 86, ocultarlo del cliente automáticamente.

12 · Panel Superadmin (superadmin.html)

Tamaño	2.372 líneas.
Rol	Plataforma SaaS. Gestiona los restaurantes cliente, planes, suscripciones, pagos, eventos.
Tablas	restaurants, subscription_plans, subscriptions, payments, platform_events, support_tickets.

Páginas

Página	Función
Dashboard	KPIs globales: MRR, restaurantes activos, tickets atendidos, NPS agregado.
Restaurantes	ABM restaurantes (alta, baja, cambio de plan, banner de mantenimiento).
Facturación	Listado de subscriptions y payments. Estado: activa, en mora, cancelada.
Usuarios	Listado de usuarios por restaurante y rol. Crea via Edge Function.
Configuración	Configuración global de plataforma, planes, parámetros de demo.
Reportes	Cohortes, churn, ARPU; gráfico MRRChart por mes.
Actividad	platform_events — log inmutable del SaaS.
Soporte	Inbox unificado de tickets (Gerente/Admin !' Superadmin) con asignación.

Mejoras sugeridas

- Onboarding asistido: alta de nuevo restaurante con wizard (plantillas de menú, mesas, zonas).
- Pagos automáticos de suscripción (Bancard recurrente / Stripe internacional).
- Métricas de uso por restaurante (pedidos / día, % de paneles activos) para detectar early churn.
- Hot-toggle de features por restaurante (delivery, estaciones, gerente, reservas).
- Banner global y modo “mantenimiento” por restaurante (ya está parcialmente: 031 maintenance_banner).
- Audit log de cambios sensibles (qué superadmin cambió qué plan a quién).

13 · Base de datos · esquema, RLS, migraciones

Motor	PostgreSQL 15 (Supabase).
Migraciones	73 archivos numerados en /supabase/migrations/.
Tablas centrales	~25 (operativas, SaaS y soporte).
Storage	menu-images.

Tablas principales (por dominio)

Dominio	Tablas
Tenancy	restaurants, user_roles, subscription_plans, subscriptions, payments, platform_events
Menú	menu_categories, menu_items, menu_item_extras, coupons, ingredients, recipes, stock_movements, stock_alerts
Operación	tables, orders, order_items, order_item_extras, order_status_history, waiter_calls, ratings, reservations, expenses
Caja	turnos_caja, movimientos_caja, cancelaciones_caja, quejas_sugerencias
Empleados	employee_shifts
Delivery	delivery_orders, delivery_zones, delivery_riders
Cocina	kitchen_stations, kitchen_station_categories, kitchen_station_zonas, order_item_station_log
Gerencia	shift_logs, manager_approvals, item_86_list
Compras	suppliers, supplier_contacts, supplier_purchases
Soporte	support_tickets, support_messages

Migraciones por etapa

- 001–016: bootstrap, login interno, fix sequences, storage de imágenes.
- 017–021: stock, turnos de caja, session de mesa.
- 022–029: mejoras KDS, fixes admin, mozo v1, RLS recursion (29).
- 030–044: delivery tipologías, employee_shifts, riders, kitchen_station inicial, admin_v2, mozo v2, customer fields, reservations, menu promos, waiter paid fields.
- 045–055: dev_reset, delivery_zones color, restaurant lat/Ing, tables_zona_shape, rider_pin_system, tables_pos_xy, payment_status, no_auto_release_mesa, order_items_delete_policy.
- 056–067: waiter debts & payroll, delivery_flow_v2, fix constraints, delivery_full_fix, delivery_cash_amount, tables_virtual_coords, orders_delivered_to_table.
- 068–073: superadmin_dev_tools, kitchen_stations definitivo, reservations zona + settings, caja fondo fijo, gerente_panel completo, support_chat.

Calidad del esquema

- Buen uso de FK con ON DELETE CASCADE / SET NULL.
- Snapshots de texto en columnas como customer_name, supplier_name, restaurant_name (mitiga borrados duros).
- Constraints CHECK en status, priority, category !' previenen valores inválidos.
- Faltan índices en algunas FK calientes (orders.table_id, order_items.order_id ya están, pero conviene auditar para delivery_orders y support_messages).

- No hay particionado de orders, lo cual está OK ahora pero será necesario al pasar de ~1M filas.

14 · Bugs y procesos erróneos detectados

Catálogo priorizado con el resultado del análisis del código en este árbol (no es una corrida de QA en producción). Cada hallazgo incluye archivo, descripción técnica, impacto operativo y solución sugerida.

B-01 · RLS abierta: 112 políticas con USING(true)

CRÍTICO

ARCHIVO

supabase/migrations/* (24 archivos)

DESCRIPCIÓN

Casi todas las tablas tienen políticas con USING(true) y/o WITH CHECK (true), incluso después de la 029 que arregló la recursión. En consecuencia, cualquier cliente con el anon key (que está en el frontend) puede leer/escribir filas de otros restaurantes apenas conozca un restaurant_id.

IMPACTO

Bloqueador para multi-tenant real: un restaurante podría leer pedidos de otro. También facilita scraping del menú de la competencia y manipulación de orders por terceros.

SOLUCIÓN SUGERIDA

Reescribir políticas: filtrar por (restaurant_id = (SELECT restaurant_id FROM user_roles WHERE user_id = auth.uid())) Crear vistas/RPC SECURITY DEFINER solo para endpoints públicos (/menu/restaurant/

B-02 · PIN del rider en texto plano + sin rate limiting

CRÍTICO

ARCHIVO

public/delivery-rider.html · supabase/migrations/20260520_051_rider_pin_system.sql

DESCRIPCIÓN

rider_pin se guarda como TEXT y se compara directo con eq("rider_pin", trimmed). Si alguien obtiene el anon key (está en el HTML), puede iterar PINs vía REST hasta encontrar un match.

IMPACTO

Cualquiera con el link público podría loguearse como rider y aceptar/"entregar" pedidos ajenos.

SOLUCIÓN SUGERIDA

Sustituir por RPC SECURITY DEFINER rider_login(pin) que valide con pgcrypto.crypt + intentos fallidos por IP (tabla rider_login_attempts). Considerar PIN de 6 dígitos + bloqueo por 15 min tras 5 fallos.

B-03 · Mesa ? / Mesa — en órdenes con table_id NULL

ALTO

ARCHIVO

public/index.html · public/mozo.html · public/caja.html

DESCRIPCIÓN

En dbSubmitOrder, si _initTableUUID() no resolvió aún, table_id se inserta como null. Después varios paneles muestran "Mesa ?" porque hacen orders.tables?.number y no hay relación.

IMPACTO

Mozo/caja no saben a qué mesa pertenece el pedido. En takeaway y delivery es esperado, pero en consumo en sitio rompe la UX.

SOLUCIÓN SUGERIDA

Bloquear el botón "Confirmar pedido" hasta que _tableUUID esté resuelto, o leer la mesa por TABLE_NUM + restaurant_id sincrónicamente antes de insertar. Fallback: rellenar table_id en background si quedó null en una orden marcada dine_in.

B-04 · Cobro puede salir 0 si order.total = 0

ALTO

ARCHIVO

public/caja.html (CobroModal)

DESCRIPCIÓN

Hoy ya hay un workaround (calcular totalReal desde items si order.total es 0). Pero el “bug raíz” es que en algunos flujos orders.total se guarda como 0 inicialmente (se actualiza recién al cobrar). Si los items aún no llegaron a CobroModal en el useEffect, totalReal puede caer a 0 si se confirma en menos de un tick.

IMPACTO

Riesgo de cobrar 0 si el cajero hace clic muy rápido. Pérdida directa de ingresos.

SOLUCIÓN SUGERIDA

No abrir CobroModal hasta que los items estén cargados (botón Confirmar deshabilitado mientras items.length === 0 && order.total === 0). Servir totals como SUM(order_items) vía vista order_totals_v.

B-05 · Anon key expuesto en HTML público

ALTO

ARCHIVO

public/config.js + todos los paneles

DESCRIPCIÓN

El anon key se entrega al navegador. Es legítimo, pero su poder depende 100% de las RLS — y como las RLS hoy son abiertas (B-01), el anon key efectivamente da acceso amplio.

IMPACTO

Se vuelve crítico solo en conjunto con B-01. Aislado, no es problema.

SOLUCIÓN SUGERIDA

Una vez B-01 resuelto, este punto cae solo. Mientras tanto, rotar el anon key si se sospecha exposición.

B-06 · tables.is_occupied desincronizado

MEDIO

ARCHIVO

public/mozo.html · public/caja.html · public/admin.html · trigger eliminado en 054

DESCRIPCIÓN

En la migración 054 se eliminó el trigger que liberaba la mesa automáticamente al marcar delivered. Ahora la mesa solo se libera con clic explícito. Pero no hay tarea de fondo que sincronice si un cajero olvidó liberarla.

IMPACTO

Mesas que aparecen ocupadas pero ya no tienen orden activa.

SOLUCIÓN SUGERIDA

Vista mv_tables_occupancy_v calculada (SELECT EXISTS ... active orders) o un cron job nocturno que libere mesas sin orders activos. Botón “sincronizar mesas” en admin.

B-07 · Race condition al insertar order_items en bucle

MEDIO

ARCHIVO

public/index.html dbSubmitOrder

DESCRIPCIÓN

Los inserts de order_items se hacen en serie con await dentro de for. Si el cliente cierra la pestaña entre el insert del order y el de los items, queda un pedido “fantasma” sin ítems. No hay transacción.

IMPACTO

Pedido aparece en cocina con 0 y sin contenido — el cajero/cocinero no sabe qué hacer.

SOLUCIÓN SUGERIDA

Mover dbSubmitOrder a un RPC create_order(n_order JSON, n_items JSON[] SECURITY DEFINER dentro

B-08 · Polling de respaldo sin backoff

MEDIO

ARCHIVO

public/index.html TrackingScreen · varios paneles

DESCRIPCIÓN

fetchStatus se llama cada 10 s siempre, también cuando ya hay realtime activo. En 50 clientes simultáneos son 5 req/s solo de tracking.

IMPACTO

Costo Supabase + saturación de PostgREST.

SOLUCIÓN SUGERIDA

Polling solo si subscribe entra a CHANNEL_ERROR/TIMED_OUT. Incremento exponencial 10s ! 20s ! 40s.

B-09 · Bookings (reservations) sin solapamiento controlado

MEDIO

ARCHIVO

supabase/migrations/20260520_040_reservations_table.sql

DESCRIPCIÓN

No hay constraint UNIQUE ni exclusión que impida reservar la misma mesa, mismo horario, dos veces. Tampoco un check de capacity (guests > capacity).

IMPACTO

Dos reservas sobre la misma mesa al mismo horario; mozo se entera al llegar el cliente.

SOLUCIÓN SUGERIDA

EXCLUDE USING gist con tstzrange(reservation_date+time, +90min) por table_id. CHECK guests <= 12 (o

B-10 · Login redirige por rol al cargar pero NO valida vencimiento

MEDIO

ARCHIVO

public/login.html

DESCRIPCIÓN

getSession().then redirige si hay sesión. No invalida si el rol cambió en el servidor (por ej. un usuario que ya fue dado de baja pero su token aún no expiró).

IMPACTO

Empleado despedido puede entrar mientras dure el token.

SOLUCIÓN SUGERIDA

En la redirección validar también que user.roles.is_active = true. Idealmente RPC get_my_profile devuelve

B-11 · Toasts duplicados / paneles cierran modales con click fuera sin confirmar

BAJO

ARCHIVO

admin.html · caja.html · superadmin.html

DESCRIPCIÓN

En varios modals, el overlay no bloquea click-outside cuando hay datos sin guardar. El usuario pierde lo escrito.

IMPACTO

Frustración del cajero/admin al cargar un proveedor largo.

SOLUCIÓN SUGERIDA

B-12 · Fórmulas inline en cobro de delivery

BAJO

ARCHIVO

public/caja.html CobroModal

DESCRIPCIÓN

cashChangeNum = cashAmountNum - (totalReal + deliveryFeeNum). Si delivery está pagado con tarjeta y el efectivo es solo de delivery, el cálculo se mezcla.

IMPACTO

Cambio mal calculado en pagos mixtos delivery + efectivo.

SOLUCIÓN SUGERIDA

Modelar pagos como array (orders, l' order, payments) con monto, método, ref. Una fila por método.

B-13 · Comentarios obsoletos en docs/auditoria_contexto_ia.md

BAJO

ARCHIVO

docs/auditoria_contexto_ia.md

DESCRIPCIÓN

Lista 8 bugs críticos y 4 ya están parcial/totalmente resueltos (tracking realtime sí está; create-user ya es Edge Function; reservas migradas; logout caja con cierre).

IMPACTO

Falsa percepción del estado.

SOLUCIÓN SUGERIDA

Reescribir el archivo (o usar este PDF como nueva línea base) y dejar solo los bugs vigentes.

15 · Cómo descubrir más bugs · metodología

Una auditoría estática (como esta) cubre código y patrones. Para llegar al 95 % hay que combinarla con pruebas dinámicas y telemetría. Recomendación de ejecución, en orden:

A · Pruebas manuales guiadas (1–2 días)

- Script de QA por panel: 10 caminos felices + 10 caminos rotos por panel (ver Apéndice 19 / Roadmap).
- Prueba “tres celulares + caja + cocina” simultánea — descubre la mayoría de bugs de realtime y mesa.
- Test de carga manual: 50 pedidos en 15 minutos. Mide latencia y Supabase rate limit.
- “Día completo de servicio en laboratorio”: abrir turno !' vender 30 órdenes !' cerrar turno !' arquear.

B · Linting y análisis estático (1 día)

- Pasar ESLint con un config básico (no-unused-vars, no-undef, react/no-array-index-key) — el código no tiene linting hoy.
- Análisis SQL: ejecutar supabase db lint o pg_lint para detectar índices faltantes y políticas RLS abiertas.
- Auditoría de dependencias CDN (versiones, integrity).

C · Telemetría en producción (continuo)

- Sentry (o LogRocket / Highlight) — captura runtime errors. Hoy todos los catch silencian con console.warn.
- Supabase logs !' tabla error_log con ON ERROR triggers en RPCs críticos.
- Cron diario que compare orders.total vs SUM(order_items.total_price) — detecta B-04 antes de que cobren ²0.
- Dashboards de salud: % pedidos con table_id null, mesas marcadas ocupadas sin orden activa, ratings sin moderar.

D · Pruebas automatizadas (medio plazo)

- Playwright headless: 1 test por flujo crítico (qr !' pedido !' cocina !' cobro). Corre en CI antes de cada deploy.
- Snapshot tests para UI clave (cliente QR Tracking, cocina TicketCard).
- pgTAP para RLS — verificar que un cliente con restaurant_id "" no ve filas ajenas.

E · Auditoría de seguridad (antes de salir a producción)

- Pentest dirigido a anon key + REST Supabase: intentar escalar privilegios.
- Revisión de Edge Function: confirmar que el JWT viene firmado y no aceptar tokens locales.
- Headers HTTP (CSP, X-Frame-Options, Permissions-Policy) — no hay vercel.json header config hoy.

16 · Seguridad · riesgos y plan

Top 3 riesgos hoy

R-1 RLS abierta (USING(true))

Es el bloqueador #1 para vender a otros restaurantes. Hasta que se cierre, el modelo es “un solo restaurante a la vez”.

R-2 PIN del rider en texto plano

Permite que cualquier persona se haga pasar por un rider y manipule entregas.

R-3 Sin políticas de headers HTTP

vercel.json no fija CSP/X-Frame/HSTS. Susceptible a clickjacking y XSS si algún día se sube contenido de terceros.

Plan de mitigación

- Sprint Seguridad #1 (1 semana): cerrar RLS por restaurante. Tests pgTAP por tabla. Migración 074 reemplaza políticas abiertas.
- Sprint Seguridad #2 (3 días): rider_login RPC con hash + intentos por IP. Migración 075.
- Endurecer Vercel: Content-Security-Policy: default-src 'self'; img-src https: data:; script-src 'self' cdn.jsdelivr.net unpkg.com; connect-src https://*.supabase.co.
- Auditar create-user.js: rate limit por IP, validar que el JWT esté firmado contra el JWT secret real (hoy solo decodifica payload).
- Rotación automática de service_role (en Vercel env) cada N días.
- 2FA opcional para superadmin (TOTP).
- Backup automático Supabase + ejercicio de restore semestral.

17 · Próximas integraciones

Lista priorizada por impacto operativo, dificultad y dependencia con otras piezas.

Pagos (Paraguay)

Integración	Uso	Dificultad
Bancard Marketplace / Vpos	Cobro con tarjeta en caja + delivery. Captura, anulación, conciliación.	Alta — KYC + entorno de pruebas + webhook.
Bancard QR (Aqua)	QR dinámico en caja y cliente (pagás escaneando con app de banco).	Media.
Tigo Money	Billetera móvil más usada en PY. Cobro directo y delivery.	Media.
Pago Móvil (BNF / Ueno)	Alternativa para mercado popular.	Media.
Stripe / Mercado Pago	Solo para suscripciones del SaaS (superadmin !' cobro a restaurantes).	Baja — fácil pero requiere KYC SaaS.

Apps de delivery

- PedidosYa / Bolt Food / Rappi — la integración formal exige certificación. Mientras tanto, webhook genérico (POST /api/intake-order) que cualquier marketplace pueda llamar para crear orden en Mythos. Esto desbloquea integradores como SocketFood o ChefDigital.
- Menú maestro exportable a JSON estándar (Open Food Format) para subir a marketplaces.

Google APIs

- Maps Places Autocomplete !' exacto en delivery-cliente.
- Geocoding / Reverse Geocoding !' traducir lat/lng a direcciones legibles para el rider.
- Distance Matrix !' ETA realista con tráfico.
- Directions !' ruta multi-pedido para rider.
- Sheets !' exportar reportes financieros al Sheet del contador.
- Google Reviews !' traer ratings públicos al panel de marketing.

Mensajería

- Twilio WhatsApp Business API !' confirmación de pedido, recordatorio de reserva, encuesta post-servicio.
- Alternativa Paraguay: WhaTicket / Wapp / Z-API si Twilio no aprueba el número.
- Emails transaccionales con Resend / Postmark !' factura electrónica, reset de password.
- SMS de respaldo (Twilio SMS o Infobip) cuando no hay WhatsApp.

Hardware / POS

- Impresión ESC/POS por LAN (Epson TM-m30, Bematech): puente Node local o QZ Tray. Sin esto, el flujo papel no funciona.
- Cajón monedero (apertura por trigger del impresor).
- Lector de QR físico para cobro Aqua (Bancard).

- Balanza para items por peso (ej. tenedor libre).

Facturación electrónica Paraguay

- SIFEN (DNIT) — emisión de factura electrónica. Existen integradores como FactSet PY, Marangatu, Bilatu.
- Reglas de timbrado, RUC, kude para factura.
- Pendiente legal: validar fechas de obligatoriedad por categoría de contribuyente.

Productividad

- Slack / Discord webhook para notificar al equipo: nuevo rating <3, queja de cliente, mesa con espera >30 min.
- Calendar (Google Calendar) !' sincronizar reservas con el calendario del salón.
- iCal feed por restaurante para que el dueño suscriba sus reservas.
- Notion / Airtable export — reportes semanales auto-enviados.

IA / Asistentes

- Resumen diario por IA (Claude API) enviado al admin: 3 highlights, 3 alertas, top 3 platos.
- Sugerencia de menú dinámico (qué promo levantar hoy según clima/día).
- Asistente de quejas: clasificar automáticamente (servicio, cocina, ambiente) y rutear al gerente.
- Voz a texto para que el mozo dicte observaciones (Web Speech API + Whisper fallback).

18 · Mejoras · cambios · obsoletos

Para QUITAR (obsoleto / muerto)

- Mesa_App.html en la raíz — versión vieja monolítica (110 KB), reemplazada por public/. Borrar tras backup.
- design.gz / design_extracted/ — assets de diseño antiguos, no se referencian.
- public/diag.html — herramienta de diagnóstico pequeña. Mover a /tools/ y proteger con login si va a quedar en producción.
- Comentarios console.error sin manejo en 88 puntos: o reemplazar por Sentry, o limpiar.
- Migraciones duplicadas con mismo número (036, 051, 069 tienen dos archivos) — renombrar para evitar confusión.

Para REEMPLAZAR

- Babel Standalone en producción !' preprocesar con esbuild antes de subir a Vercel (compatible con el patrón CDN: solo cambia el script src).
- Comentarios de tipos sueltos !' migrar gradualmente a JSDoc con //@ts-check para detección temprana.
- localStorage del tema !' preference en user_profile (sync entre dispositivos).
- SplitBillModal del cliente !' mover al mozo / caja (es donde más se usa en la práctica).

Para MEJORAR (sin reescribir)

- Centralizar helpers fmt/fmtTime/SL/SC en /public/lib/format.js — hoy duplicados.
- design-system.css ya existe pero está infrautilizado: empujar más estilos hacia ahí.
- Loader / skeleton consistente en todos los paneles (hoy varía panel a panel).
- Modo oscuro completo: muchos paneles ya tienen toggle pero algunos pantallazos quedan claros.
- Onboarding del cliente nuevo: tooltip “bienvenido a La Huaca”, animación de marca.
- Compresión de imágenes con browser-image-compression antes de subir a Storage.
- Caching de menu con stale-while-revalidate (1 carga, fondo refresh, fallback a localStorage).
- Versión PWA (manifest.json + service worker) — al menos para el cliente QR.

Para AÑADIR (oportunidades de producto)

- Loyalty: puntos por pedido, niveles, recompensas. Tabla loyalty_accounts + loyalty_movements.
- Encuesta NPS automática 24h post-visita por WhatsApp.
- Reservas con depósito (señal) — bloquea no-shows.
- Catering / eventos: tipo de orden “evento”, hoja de cálculo, anticipo.
- Receta visual para el cocinero: foto del plato + pasos en la TicketCard al hacer “preparando”.
- Modo Sala Privada: bloquear mesas para evento, marcar “grupo de N personas” !' vista especial en mozo.

19 · Roadmap a producción real

Sprint 1 (semana 1) · seguridad y datos

- Cerrar RLS por restaurante (B-01) con tests pgTAP. Migración 074.
- Rider login con hash + rate limit (B-02). Migración 075.
- Headers HTTP en vercel.json (CSP, HSTS).
- Rotar anon key y service_role; revisar SENSITIVE_DATA.md.

Sprint 2 (semana 2) · estabilidad operativa

- RPC create_order transaccional (resuelve B-04 y B-07).
- Sincronización de tables.is_occupied (B-06) con vista calculada.
- Cron diario de validaciones: orders huérfanos, totales 0, mesas inconsistentes.
- Snapshot tests Playwright para 5 flujos (qr, delivery, cocina, mozo, caja).

Sprint 3 (semana 3) · pagos

- Integración Bancard Vpos (cobro tarjeta caja).
- Integración Bancard QR (cliente).
- Integración Tigo Money (opcional, prioridad media).
- order_payments table + UI mixta correcta (resuelve B-12).

Sprint 4 (semana 4) · operaciones físicas

- Impresión ESC/POS: ticket cocina + ticket cliente.
- SIFEN factura electrónica (al menos sandbox).
- Apertura cajón monedero por trigger del impresor.

Sprint 5 (semana 5) · CRM y comunicación

- Tabla customers + upsert al cerrar pedido.
- WhatsApp Business: confirmación de pedido + recordatorio de reserva.
- Encuesta NPS 24 h después de la visita.

Sprint 6 (semana 6) · multi-restaurante real

- Onboarding wizard en superadmin: alta restaurante + plantillas.
- Feature flags por restaurante (delivery on/off, gerente on/off, ...).
- Hot toggle de banner de mantenimiento por restaurante (ya está parcial — terminar).
- Métricas de uso por restaurante para early churn detection.

Sprint 7+ (opcional) · diferenciación

- App rider nativa (Capacitor) + push notifications.
- PWA cliente con offline catalog.
- IA: resumen diario del dueño + asistente de queja.
- Integración apps delivery (PedidosYa / Bolt / Rappi).

20 · Crítica final y % de completitud

Crítica honesta (lo que diría un revisor externo)

Mythos demuestra ambición y oficio raros en una v1: cubre un dominio enorme con poco código de andamiaje, mantiene UX consistente entre nueve paneles y resuelve cuestiones difíciles como turnos de caja, kitchen stations y multi-tenant SaaS. Su mayor virtud — la simplicidad CDN + Supabase — es también su mayor limitación cuando empiezan los problemas de escala: sin RLS estricta, sin transacciones explícitas, sin tests, todo descansa en disciplina humana al revisar. El siguiente paso natural no es reescribir, sino reforzar cimientos: seguridad, pagos, telemetría.

Lo que está MUY BIEN

- Cobertura de roles y casos de uso — solo le falta operador de central telefónica.
- Panel de caja con turnos, fondo fijo, cancelaciones, quejas — nivel de detalle poco común en v1.
- Sistema de cocina por estaciones con token compartible — apenas hay competidores PY con eso.
- Soporte chat embebido (Gerente/Admin !” Superadmin) — pieza pro de un SaaS maduro.
- Migraciones bien numeradas y comentadas, fáciles de auditar.
- Decisión de Edge Function para gestión de usuarios — corrigió un riesgo crítico.

Lo que está MUY MAL

- RLS de 112 USING(true). Esto, hoy, hace que “multi-tenant” sea solo de fachada.
- Sin pagos reales. El cliente “elige método” y se confía en el cajero — esto no escala.
- PIN rider sin hash + sin rate limit.
- Cero tests. Cualquier refactor importante es caminar a oscuras.
- Documentación parcialmente desactualizada (auditoria_contexto_ia.md).

Veredicto numérico — ponderación por área

Área	Peso	Madurez	Contribución
Funcionalidad de paneles	25%	85%	21.25
Base de datos y migraciones	15%	80%	12.00
UX visual	10%	85%	8.50
Multi-tenancy real (RLS)	10%	30%	3.00
Seguridad (PIN, headers, JWT)	7%	35%	2.45
Pagos reales	8%	0%	0.00
Integraciones externas	7%	5%	0.35
CRM / Clientes	5%	10%	0.50
Testing y telemetría	5%	0%	0.00
Documentación operativa	4%	60%	2.40
Hardware POS / impresión	4%	0%	0.00
Total ponderado	100%	—	50.45

La suma ponderada estricta da ~50 % de “producto SaaS comercial

completo". La razón por la que el veredicto subjetivo del resumen ejecutivo es más alto (~70 %) es que las áreas con peso real para un piloto en un solo restaurante (funcionalidad + DB + UX) están bien resueltas. Cuando el objetivo cambia a "vender a 50 restaurantes", las áreas con 0-35 % de madurez se vuelven bloqueantes y el número correcto es el ponderado.

Mi recomendación

Próxima decisión estratégica

1. Si el plan es PILOTO en La Huaca (un único restaurante): el sistema está listo, salvo Bancard. Cuatro semanas de trabajo a ritmo normal lo dejan en producción real.
2. Si el plan es SaaS multi-restaurante: hay ~10 semanas de trabajo serio antes de salir a vender. La mitad de ese tiempo es seguridad y multi-tenant; la otra mitad pagos, impresión y CRM.

En ambos casos, antes de la primera venta externa: cerrar B-01, B-02, B-03, B-04 y B-07. Son los cinco puntos que separan "demo robusta" de "producto operativo confiable".

Fin del informe.

